

Лабораторная работа № 2 по курсу дискретного анализа: сбалансированные деревья

Выполнил студент группы 08-207 МАИ *Дружинин Данила*.

Условие

1. В данной лабораторной работе требовалось создать программную библиотеку, реализующую указанную структуру данных, на основе которой разработать программу-словарь. Словарь должен хранить пары вида «ключ–значение», где ключом является регистронезависимое слово, состоящее из букв английского алфавита длиной не более 256 символов, а значением — целое беззнаковое число из диапазона от 0 до $2^{64} - 1$. Программа должна обрабатывать входные данные построчно до конца файла и поддерживать основные операции словаря: добавление нового слова, удаление существующего слова и поиск слова с выводом связанного с ним значения.
2. Задача 1. 1 AVL-дерево

Метод решения

В данной работе реализован словарь на основе сбалансированного двоичного дерева поиска (AVL-дерева). Структура данных хранит пары «ключ–значение», где ключом является строка фиксированной длины, а значением — число типа `uint64_t`.

Каждый узел дерева содержит:

- строковый ключ;
- числовое значение;
- высоту узла;
- указатели на левое и правое поддеревья.

Дерево удовлетворяет свойству бинарного дерева поиска: для каждого узла все ключи в левом поддереве меньше текущего, а в правом — больше.

Балансировка дерева осуществляется с использованием высот поддеревьев. Для каждого узла вычисляется разность высот левого и правого поддеревьев. Если эта разность выходит за пределы допустимого значения, выполняются повороты дерева (левый или правый, а также их комбинации) для восстановления баланса.

Основные операции реализованы рекурсивно.

Поиск выполняется аналогично поиску в бинарном дереве поиска: на каждом шаге производится сравнение строк, и переход осуществляется в левое или правое поддерево.

Вставка производится следующим образом. Если текущий узел отсутствует, создаётся новый узел. В противном случае выполняется сравнение ключей:

- если ключ уже существует, вставка не производится;
- если ключ меньше текущего, вставка выполняется в левое поддерево;
- если больше — в правое.

После вставки пересчитывается высота узла и при необходимости выполняется балансировка дерева с помощью поворотов.

Удаление также реализовано рекурсивно. При нахождении узла:

- если у узла отсутствует один из потомков, он заменяется другим поддеревом;
- если оба потомка существуют, выбирается узел с минимальным ключом в правом поддереве, после чего выполняется замена и рекурсивное удаление.

После удаления производится обновление высот и балансировка дерева.

Для обеспечения регистронезависимости ключей все входные строки приводятся к нижнему регистру. Для работы со строками используются собственные функции сравнения и копирования.

Обработка входных данных осуществляется построчно. В зависимости от команды вызываются операции вставки, удаления или поиска.

Реализованная структура обеспечивает выполнение операций вставки, поиска и удаления за логарифмическое время.

Описание программы

Программа реализована в одном исходном файле на языке C++ и представляет собой консольное приложение. Для работы используются стандартные библиотеки языка для ввода-вывода, работы с типами данных и файлами.

Основная структура данных

Ключевым элементом программы является структура узла дерева:

- **TNode** — структура, описывающая узел AVL-дерева. Она содержит:
 - массив символов фиксированного размера для хранения ключа;
 - значение типа `uint64_t`;
 - поле `Height`, хранящее высоту поддерева;
 - указатели на левое и правое поддерева (`Left`, `Right`).

Класс словаря

Для работы с деревом реализован класс `TavlDictionary`, который содержит указатель на корень дерева (`Root`) и предоставляет интерфейс для выполнения операций со словарём:

- **Insert** — добавление нового элемента. Возвращает признак успешности вставки;
- **Find** — поиск элемента по ключу с возвратом соответствующего значения;
- **Erase** — удаление элемента по ключу;
- **Clear** — удаление всех узлов дерева;
- **Save** — сохранение текущего состояния словаря в бинарный файл;
- **Load** — загрузка словаря из файла с заменой текущего состояния.

Реализация дерева

Работа дерева обеспечивается набором вспомогательных функций:

- **InsertNode** — рекурсивная вставка узла с последующей балансировкой;
- **FindNode** — поиск узла по ключу;
- **EraseNode** — удаление узла с последующей балансировкой;
- **FindMin** — поиск узла с минимальным ключом в поддереве;
- **RemoveMin** — удаление узла с минимальным ключом;
- **ClearTree** — рекурсивное удаление всех узлов дерева.

Балансировка дерева

Для поддержания сбалансированности используются следующие функции:

- **GetHeight** — получение высоты узла;
- **FixHeight** — пересчёт высоты узла на основе высот поддеревьев;
- **BalanceFactor** — вычисление разности высот левого и правого поддеревьев;
- **RotateLeft, RotateRight** — повороты дерева;
- **Balance** — приведение узла к сбалансированному состоянию.

Работа со строками

Поскольку стандартные строковые типы не используются, реализованы собственные функции:

- **StrLen** — вычисление длины строки;
- **StrCopy** — копирование строки;
- **StrCompare** — лексикографическое сравнение строк;

- `ToLowerWord` — приведение строки к нижнему регистру.

Работа с вводом данных

Для разбора входных данных используются функции:

- `ReadWord` — чтение слова из строки;
- `ReadUInt64` — чтение числового значения;
- `SkipSpaces` — пропуск пробельных символов;
- `RemoveNewline` — удаление символов конца строки.

Сохранение и загрузка

Для работы с файлами реализованы функции:

- `SaveNodes` — последовательная запись узлов дерева в файл;
- `LoadNodes` — чтение узлов из файла и восстановление дерева;
- `CountNodes` — подсчёт количества узлов;
- `ReadExact` — вспомогательная функция для чтения фиксированного количества данных.

Данные сохраняются в бинарном формате, включающем количество элементов и последовательность пар «ключ–значение».

Главная функция

Функция `main` выполняет построчное чтение входных данных, определяет тип команды (вставка, удаление, поиск, сохранение или загрузка) и вызывает соответствующие методы класса словаря. Результаты выполнения операций выводятся в стандартный поток вывода.

Дневник отладки

В процессе разработки программы было выполнено несколько итераций исправлений, связанных как с логикой работы AVL-дерева, так и с обработкой входных данных и файлов.

1. Ошибка балансировки после удаления

На раннем этапе реализации при удалении элементов дерево переставало оставаться сбалансированным. Проблема заключалась в том, что после рекурсивного удаления не выполнялась корректная балансировка узлов.

Для устранения ошибки в функцию удаления была добавлена обязательная балансировка узла после изменения поддеревьев, а также пересчёт высоты.

2. Некорректная обработка регистра ключей

Изначально слова сравнивались без приведения к единому регистру, из-за чего строки вида "а" и "А" считались разными ключами.

Проблема была решена добавлением функции приведения строки к нижнему регистру перед выполнением всех операций со словарём.

3. Ошибка при повторной вставке элемента

В начальной версии при попытке вставки уже существующего ключа значение перезаписывалось, что противоречило условию задачи.

Ошибка была исправлена добавлением проверки на равенство ключей в функции вставки и возвратом признака, запрещающего повторную вставку.

4. Ошибка удаления узла с двумя потомками

При удалении узла с двумя поддеревьями происходила потеря части дерева. Причиной было некорректное переподключение поддеревьев.

Исправление заключалось в использовании узла с минимальным ключом в правом поддереве для замены удаляемого узла, а также корректной реализации функции удаления минимального элемента.

5. Проблемы при работе с бинарным файлом

На этапе реализации сохранения и загрузки словаря возникали ошибки при чтении данных из файла, приводящие к некорректному восстановлению дерева.

Проблема была решена добавлением строгой проверки количества прочитанных байт, а также контролем корректности длины ключа перед его чтением.

6. Обработка пустого и отсутствующего файла при загрузке

Изначально программа завершалась с ошибкой при попытке загрузить несуществующий файл.

В дальнейшем была добавлена обработка данного случая: если файл отсутствует или пуст, словарь очищается, а операция считается успешной.

7. Ошибка при разборе входных строк

На ранних этапах возникали ошибки при чтении команд из-за наличия лишних пробелов и символов перевода строки.

Для исправления были реализованы функции пропуска пробелов и удаления символов конца строки, что обеспечило корректный разбор входных данных.

В результате проведённой отладки программа корректно обрабатывает все предусмотренные операции и проходит тестирование на различных наборах входных данных.

Тест производительности

Для оценки производительности реализованной структуры данных был проведён эксперимент, в рамках которого измерялось время выполнения программы при обработке различных объёмов входных данных.

В ходе тестирования для каждого значения n выполнялась последовательность операций:

- вставка n элементов;
- поиск n элементов;
- удаление n элементов.

Таким образом, суммарно выполнялось порядка $3n$ операций над AVL-деревом. Результаты измерений представлены в таблице:

n	время (с)
10000	0.020
20000	0.032
30000	0.064
50000	0.082
70000	0.137
100000	0.254
150000	0.382
200000	0.475
300000	0.665
400000	0.828
500000	1.264
700000	1.836
1000000	2.769
1500000	4.313

На основе полученных данных был построен график зависимости времени выполнения программы от количества операций.

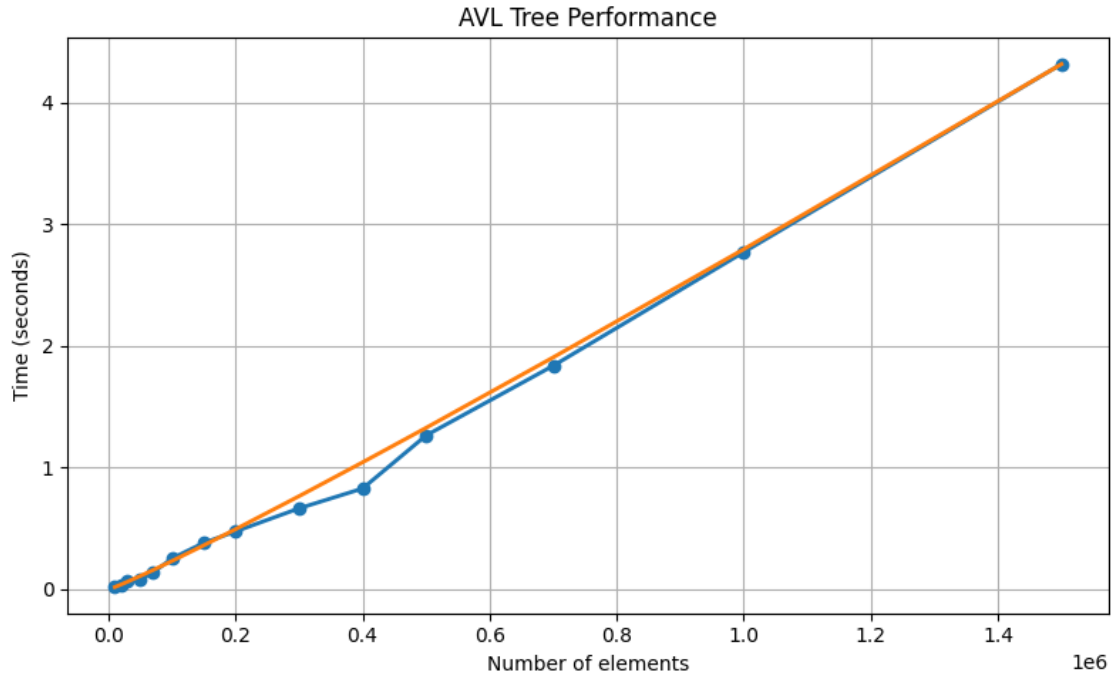


Рис. 1: Сравнение экспериментального времени работы программы с теоретической зависимостью $n \log n$.

Анализ графика показывает, что экспериментальная зависимость времени выполнения близка к функции $n \log n$. При увеличении объёма входных данных наблюдается рост времени, превышающий линейный, что объясняется логарифмической составляющей сложности операций в сбалансированном дереве.

Поскольку каждая операция (вставка, поиск, удаление) в AVL-дереве выполняется за $O(\log n)$, суммарное время выполнения последовательности из n операций имеет сложность $O(n \log n)$.

Таким образом, экспериментальные результаты согласуются с теоретической оценкой сложности алгоритмов, реализованных в программе.

Недочёты

В ходе разработки программы были выявлены некоторые ограничения и потенциальные недостатки реализации.

Во-первых, при работе с памятью используется динамическое выделение памяти (оператор `new`) без явной обработки исключений. В случае нехватки памяти программа может завершиться аварийно, так как обработка ошибки выделения памяти не реализована.

Во-вторых, реализация операций сохранения и загрузки словаря предполагает корректность формата входного файла. Несмотря на наличие базовых проверок, при по-

вреждении файла возможны ситуации, когда загрузка завершается неуспешно без детальной диагностики причины ошибки.

Также стоит отметить, что в реализации используется рекурсивный подход для операций вставки, удаления и обхода дерева. При теоретически возможной глубине рекурсии это может привести к переполнению стека, однако в сбалансированном AVL-дереве такая ситуация маловероятна.

Кроме того, сравнение строк выполняется посимвольно, что может оказывать влияние на производительность при работе с длинными ключами (до 256 символов), однако в рамках поставленной задачи это не является критичным.

В целом программа корректно реализует требуемую функциональность и проходит тесты, однако указанные особенности могут быть учтены при дальнейшем улучшении реализации.

Выводы

В ходе выполнения лабораторной работы была реализована структура данных «словарь» на основе AVL-дерева, обеспечивающая эффективное хранение и обработку пар «ключ–значение».

Данная структура может применяться в задачах, где требуется поддерживать упорядоченные данные с быстрым поиском, вставкой и удалением элементов. К типовым областям применения относятся:

- реализация словарей и ассоциативных массивов;
- обработка текстовых данных (поиск слов, подсчёт частот);
- хранение и обработка индексированных данных;
- системы, требующие гарантированного времени выполнения операций.

Все основные операции (поиск, вставка, удаление) в AVL-дереве выполняются за $O(\log n)$, что обеспечивается за счёт поддержания сбалансированности дерева. Экспериментальные результаты показали, что суммарное время выполнения последовательности операций соответствует оценке $O(n \log n)$. По памяти реализованная структура данных имеет сложность $O(n)$, так как для хранения каждого элемента словаря создаётся отдельный узел дерева. Каждый узел содержит ключ фиксированного максимального размера, значение, а также указатели на дочерние элементы и вспомогательные данные (высоту поддерева).

Реализация данной структуры данных требует аккуратного подхода, так как необходимо корректно поддерживать высоту узлов и выполнять балансировку дерева при вставке и удалении элементов. Наибольшую сложность представляет реализация операции удаления, связанная с перестроением структуры дерева и сохранением его свойств.

В процессе разработки были решены задачи корректной балансировки дерева, обработки строковых ключей и организации эффективного хранения данных. Полученная программа успешно проходит тестирование и соответствует поставленным требованиям.